

2. Back-End

Instalación y configuración del componente administrativo de Alerta Satipo

2.1. Alcance y tecnología implementada

El componente administrativo implementa una arquitectura hexagonal en JavaScript: dominio independiente, casos de uso, puertos de entrada y salida y adaptadores HTTP y Supabase. El panel consulta la API administrativa; el adaptador de persistencia accede a PostgreSQL mediante el SDK de Supabase conservando el JWT del usuario y RLS. El mismo núcleo se ejecuta en un servidor Node.js local o en Supabase Edge. No se utiliza Express: HTTP se adapta mediante Request/Response y node:http.

Este apartado explica cómo instalar y verificar el componente presente en el repositorio alerta-satipo. La instalación de Flutter del apartado 1 y el servicio WeatherSatipo en C#.NET del apartado 4 corresponden a otros componentes: su comunicación con este panel no se considera verificada por estos pasos. No se debe configurar ni ejecutar un servicio .NET para seguir este procedimiento.

La página está destinada a administradores previamente autorizados y no ofrece registro público. Firebase Hosting publica sus archivos estáticos; Supabase conserva los datos. La estructura detallada de las tablas se desarrolla en el apartado 3, Base de Datos.

La evidencia de arquitectura está en backend/domain, backend/application/ports.mjs, backend/application/admin-service.mjs, backend/infrastructure y backend/bootstrap.mjs. El núcleo no importa Supabase ni HTTP. Los adaptadores se inyectan por solicitud; las pruebas verifican dependencias, reglas y casos de uso sin red. El alcance migrado comprende lecturas, operaciones demostrativas, incidentes y configuración del panel; Auth y perfil conservan su integración directa con Supabase.

2.2. Requisitos previos

- Windows con PowerShell, Git y acceso autorizado al repositorio privado.
- Node.js 22 actualizado a una revisión reciente, con npm. El comando de configuración utiliza --env-file-if-exists.
- Python 3.10 o superior para el servidor de desarrollo. No se requieren paquetes Python adicionales para iniciar el panel.
- Visual Studio Code u otro editor, navegador e Internet.
- URL y clave pública del proyecto Supabase; una cuenta administrativa del aplicativo para ingresar.

Comprobar las herramientas en una terminal nueva:

```
git --version
node --version
npm --version
python --version
```

Resultado esperado: los cuatro comandos muestran una versión instalada. Si PowerShell bloquea npm.ps1, utilizar npm.cmd en los comandos siguientes. No es necesario instalar PostgreSQL local ni Docker para conectarse al Supabase alojado.

2.3. Descargar el proyecto e instalar dependencias

Abrir PowerShell en una carpeta de trabajo y ejecutar:

```
git clone https://github.com/76778920-tech/alerta-satipo.git
Set-Location alerta-satipo
npm ci
npm run build
```

La cuenta de GitHub debe tener acceso al repositorio. Si ya existe una copia del proyecto, abrir esa carpeta y conservar los cambios locales antes de actualizarla; no es necesario clonar de nuevo.

npm ci instala las dependencias fijadas por package-lock.json. npm run build genera shared/vendor/supabase.js. Resultado esperado: mensaje «SDK de Supabase compilado localmente». Mantener la terminal en la raíz del repositorio para los pasos posteriores.

2.4. Configurar la conexión con Supabase

Comprobar si existe el archivo de configuración local:

```
Test-Path .env
```

Solo si el resultado es False, copiar la plantilla:

```
Copy-Item .env.example .env
```

Editar .env y configurar el entorno. Para utilizar el proyecto compartido:

```
SATIPO_MODE=supabase
SUPABASE_URL=https://ddfmooylklgwnrmlxqzc.supabase.co
SUPABASE_PUBLISHABLE_KEY=<clave_publica_del_proyecto>
SUPABASE_SECRET_KEY=
SATIPO_API_URL=http://127.0.0.1:8787/api
```

Reemplazar el marcador por la clave publishable o anon correspondiente al mismo proyecto. La plantilla viene en modo demo; se debe cambiar a supabase. Para una base independiente se sustituye también la URL. El enlace CSV de GitHub no se utiliza como SUPABASE_URL: es la fuente de los datos importados.

La clave privada no es necesaria para ejecutar el panel. SUPABASE_SECRET_KEY se usa únicamente en operaciones administrativas específicas y puede permanecer vacía en esta instalación. Nunca introducir una clave secret o service_role en SUPABASE_PUBLISHABLE_KEY.

```
npm run configure
```

Resultado esperado: archivo config/public.json generado y mensaje «modo supabase». Este archivo contiene la URL y la clave pública; no la clave administrativa. Volver a ejecutar el comando después de cambiar .env. No publicar .env ni archivos config/*.local.*.

2.5. Seleccionar y comprobar la base de datos

Ruta A — Proyecto existente. Las tablas y la muestra ya están cargadas. No ejecutar migraciones, seed.sql, el importador ni scripts de creación de usuarios. Continuar con la autorización del acceso y la ejecución local. Las modificaciones realizadas desde este entorno afectan a la base compartida.

Ruta B — Proyecto nuevo y vacío. El responsable debe preparar la base conforme al apartado 3. En SQL Editor, ejecutar el contenido completo de estos archivos, uno por uno y en este orden:

1. supabase/migrations/202609230001_initial.sql
2. supabase/migrations/202609230002_private_admin_helper.sql
3. supabase/seed.sql
4. supabase/migrations/202609230003_demo_operations.sql

La migración 003 requiere que las 300 lecturas existan previamente. No ejecutar las tres migraciones antes del seed. Ejecutar archivos con SQL Editor no registra automáticamente versiones en el historial de la CLI; no combinar este procedimiento con db push sin reconciliar ese historial.

Como alternativa exclusiva al paso 3, puede utilizarse `npm run import:dataset` con la clave privada configurada por el responsable. No hace falta ejecutar ambas opciones. El archivo versionado contiene las 300 lecturas; `npm run dataset` descarga y vuelve a preparar la fuente, por lo que no es un requisito de instalación.

Comprobación de lectura, desde SQL Editor:

```
select count(*) as total,
       count(*) filter (where fire_alarm) as con_alarma,
       count(*) filter (where not fire_alarm) as sin_alarma
from public.smoke_readings
where dataset_id = 'smoke-detection-iot-300-v1';

select node_id, count(*) as lecturas
from public.demo_node_readings
group by node_id order by node_id;
```

Resultado esperado: 300 registros, 214 con etiqueta de alarma y 86 sin ella; seis nodos virtuales con 50 vínculos cada uno. Son datos históricos y lotes demostrativos, no incendios actuales ni dispositivos instalados.

2.6. Autenticación y autorización administrativa

Para la base compartida se utiliza una cuenta existente autorizada por el responsable. La contraseña del aplicativo no es la contraseña de PostgreSQL ni la de GitHub. Ser integrante de la organización Supabase tampoco concede automáticamente acceso al panel.

En un proyecto nuevo, el responsable debe aprovisionar una identidad autorizada en Supabase Auth. Mantener habilitado el acceso por correo y contraseña y desactivar el registro público mediante «Allow new users to sign up». La creación administrativa inicial no equivale a habilitar un formulario de registro.

Después de obtener el UUID real de la cuenta en Authentication, asignar el permiso desde SQL Editor, únicamente si aún no lo tiene:

```
insert into public.administradores (id, display_name)
values ('UUID_REAL_DEL_USUARIO', 'Administrador Satipo')
on conflict (id) do nothing;
```

El UUID debe existir en `auth.users`; el esquema crea el perfil asociado en clientes. El panel consulta la identidad y comprueba su pertenencia a administradores. Las operaciones de base de datos se someten además a privilegios y políticas RLS.

Si se utilizará recuperación de contraseña local, permitir la URL `http://127.0.0.1:8000/shared/account.html` en la configuración de redirecciones de Supabase. Agregar la dirección de desarrollo sin reemplazar la Site URL del sitio de producción. Referencias oficiales al final de este apartado.

2.7. Iniciar y detener el entorno local

En la primera terminal, iniciar la API hexagonal con `.env` configurado:

```
npm run start:api
```

Resultado esperado: API hexagonal: `http://127.0.0.1:8787/api`. En otra terminal, desde la misma raíz, iniciar el frontend:

```
npm start
```

Resultado esperado en la terminal: «Alerta Satipo: http://127.0.0.1:8000». Mantener ese proceso activo y abrir <http://127.0.0.1:8000/shared/login.html>. Ingresar con la cuenta administrativa autorizada; el acceso correcto conduce a [/web/index.html](http://127.0.0.1:8000/web/index.html).

No abrir los HTML mediante `file://`. El panel carga configuración y recursos por HTTP y consulta Supabase mediante HTTPS. `shared/js/backend.js` mantiene la sesión y envía solicitudes a la API.

`shared/js/dataset.js` usa GET `/readings`; `web/js/operations.js` usa GET `/operations` y PATCH `/cases/:id` o `/maintenance/:id`. El caso de uso verifica autorización, valida el dominio e invoca el repositorio.

Para detener el servidor, pulsar Ctrl+C. Si el puerto 8000 está ocupado:

```
python scripts/serve.py --port 8001
```

Utilizar entonces <http://127.0.0.1:8001/shared/login.html> y autorizar la URL de recuperación con ese puerto si se necesita. Ambos servidores locales son de desarrollo y se detienen con Ctrl+C en sus terminales. El adaptador Node permite por defecto orígenes 127.0.0.1:8000 y localhost:8000; para otro puerto, agregar su origen en `API_ALLOWED_ORIGINS` y reiniciar la API. Con `SATIPO_API_URL` vacío, `npm run configure` selecciona la función Edge del proyecto y no se necesita iniciar Node local.

2.8. Verificación de conexión y pruebas

Primero comprobar el flujo sin modificar datos: ingreso administrativo, consulta de 300 lecturas, filtros de 214 etiquetas positivas y 86 negativas, seis nodos virtuales, visualización de casos y mantenimiento, y cierre de sesión. No es necesario cambiar estados compartidos para comprobar la instalación.

Ejecutar las pruebas locales:

```
npm test
```

Resultado esperado: dieciséis pruebas aprobadas. Incluyen dominio, casos de uso en memoria, concurrencia, adaptadores HTTP/Supabase y restricciones de importación, además de PostgreSQL embebido mediante PGlite. No modifican Supabase remoto ni prueban por sí solas la instalación completa en un equipo nuevo.

Para verificar la API con Postman, crear variables locales `base_url`, `publishable_key`, `admin_email`, `admin_password` y `access_token`. `base_url` debe ser la URL del proyecto Supabase configurado.

Solicitud de autenticación: POST `{{base_url}}/auth/v1/token?grant_type=password`. Cabeceras: `apikey: {{publishable_key}}` y `Content-Type: application/json`. Cuerpo:

```
{"email":"{{admin_email}}", "password":"{{admin_password}}"}
```

Resultado esperado: HTTP 200 y `access_token`. Guardarlo localmente y utilizarlo en la cabecera `Authorization: Bearer {{access_token}}`, junto con `apikey: {{publishable_key}}`, para la consulta siguiente.

Para probar el backend hexagonal, crear `api_url` con <http://127.0.0.1:8787/api> o la URL HTTPS de la función admin-api. Realizar GET `{{api_url}}/readings` con el token obtenido. La URL `base_url` se conserva como URL Supabase para autenticación; no confundirla con `api_url`. No se requieren parámetros para la muestra de 300 filas.

Resultado esperado: HTTP 200 y un arreglo de 300 filas. Este procedimiento documenta resultados esperados; no se presenta como evidencia de una ejecución ya realizada en Postman.

Conservar capturas propias de versiones, compilación, npm test y consulta correcta. Ocultar contraseñas, tokens y claves privadas. Las figuras explicativas no reemplazan esas evidencias.

2.9. Solución de problemas y cierre de la instalación

- Repositorio no encontrado: verificar invitación al repositorio privado y cuenta utilizada por Git.
- Herramienta no reconocida: revisar instalación y PATH y abrir una terminal nueva. Si solo existe py, ejecutar `py scripts/serve.py` o configurar el comando python utilizado por npm start.
- Opción `--env-file-if-exists` desconocida: actualizar Node.js a una revisión reciente compatible.
- No se pudo conectar o falta configuración: verificar `.env`, ejecutar `npm run configure` y usar el servidor desde la raíz del proyecto.
- Clave rechazada: comprobar que la clave pública y la URL pertenezcan al mismo proyecto; no usar claves administrativas en el navegador.
- Acceso denegado: comprobar credenciales de Auth, fila en administradores, perfil en clientes y proyecto configurado.
- Relaciones demo inexistentes: en una base nueva, verificar que la migración 003 se ejecutó después de cargar las 300 filas.
- Error de red o consulta vacía: revisar conectividad, sesión, proyecto y políticas RLS; no desactivar las políticas para ocultar el problema.
- Recuperación redirige a otro origen: verificar la URL autorizada `shared/account.html` y su puerto.

La instalación queda verificada cuando el servidor inicia, un administrador accede, se consultan las 300 lecturas y seis lotes, y las pruebas locales aprueban. Esto valida el componente administrativo y su separación hexagonal. No acredita integración con Flutter o WeatherSatio. Para despliegue de Edge ejecutar `npm run build:api` y desplegar admin-api con Supabase CLI; para publicar el frontend usar una API HTTPS, nunca localhost. Consultar `docs/ARQUITECTURA_IMPLEMENTADA.md` para instrucciones y límites de concurrencia.

Fuentes de implementación: `package.json`; `scripts/build.mjs`; `scripts/configure.mjs`; `scripts/serve.py`; `shared/js/backend.js`; `shared/js/dataset.js`; `web/js/operations.js`; migraciones SQL y `tests/database.test.mjs`.

Configuración oficial de autenticación: <https://supabase.com/docs/guides/auth/general-configuration>

Configuración oficial de redirecciones: <https://supabase.com/docs/guides/auth/redirect-urls>